Tapeout: Devpost submission kit

Nebius × NVIDIA Global AI Hackathon · deadline **30 Oct 2026, 10:30 PM IST** · repo github.com/Amritha902/NEBIUS · this kit is also in the repo under `submission-kit/`

0. What is in this folder

File	Use it for
`1-devpost-fields.txt`	Every short Devpost field, ready to copy
`2-devpost-description.md`	The full project description: paste it into "About the project" (Devpost accepts Markdown)
`3-Tapeout-Devpost-Submission.pdf`	This guide
`4-feedback-answers.txt`	Answers for the feedback survey (Most Valuable Feedback prize)
`images/`	7 gallery images; upload `shmoo.png` first, as the cover
`video/Tapeout-demo.mp4`	**Demo video: 2 min 12 s, 1920×1080, AI voiceover + subtitles + music.** Upload this to YouTube.
`video/voiceover-script.txt`	What the voice says, with timestamps
`docs/`	The full project report and the setup guide (PDFs)

Do these 3 things before submitting, or the links won't open for judges:

1. **GitHub Pages** (1 minute): github.com/Amritha902/NEBIUS → Settings → Pages → "Deploy from a branch" → **main**, folder **/docs** → Save. The demo is then at amritha902.github.io/NEBIUS/demo/. While in Settings → Branches, set the default branch to **main**.
2. **Claude-hosted demo**: the owner opens claude.ai/artifact/YFqwxVpe3B6LLcDgoV2MmE → **Share** → anyone with the link.
3. **YouTube**: upload `video/Tapeout-demo.mp4` as **Public** (not Unlisted, not Private). Put its link in the Video field. Upload early: processing takes a while.

1. Field by field

Devpost field	Enter
Project name	**Tapeout**
Elevator pitch	An AI chip designer that tries to prove its own datasheet wrong. Nemotron designs; a red team attacks across corners; ngspice judges. Every number links to its simulation.
About the project	Paste all of `2-devpost-description.md` (the full text is also in section 2 below)
Built with	nvidia-nemotron, nebius-token-factory, tavily, python, fastapi, ngspice, skywater-sky130, server-sent-events, docker, playwright
Track	**Best Apps and Agents**

Extra prizes	Tick **Best Use of Tavily**. Fill in the feedback survey for **Most Valuable Feedback** using `4-feedback-answers.txt`
"Try it out" links	1. https://github.com/Amritha902/NEBIUS 2. https://amritha902.github.io/NEBIUS/demo/ (after Pages is on) 3. https://claude.ai/artifact/YFqwxVpe3B6LLcDgoV2MmE (after Share is set)
Video demo link	Your public YouTube link for `video/Tapeout-demo.mp4`
Image gallery	The 7 files in `images/`, in the order shown in section 3
Open-source license	MIT. GitHub shows it at the top of the repo
Did it exist before?	Partly. Use the paragraph in `1-devpost-fields.txt`: the earlier engine is cadence_auto, and everything in `tapeout/` and `web/` is new.
City (Builders & Brews)	Only if you attended one in person; otherwise leave it blank

Be accurate about live runs. The demo video and the replays come from an **offline** recording. Built-in stand-ins replaced Nemotron there; every simulation is real, and both the page and the voiceover say so. The live Nebius path is tested against a protocol-faithful mock. If you get the Nebius key working before submitting (setup guide, Part C), record a live run and use that instead. The description already says this plainly; don't claim more than it does.

2. Project description (paste into "About the project")

Every number on the datasheet links to the exact SPICE deck and ngspice log that produced it. Tapeout writes a datasheet, then sends a red team to prove where it is false. What survives becomes the promise; what breaks becomes a published limit. Run `python -m tapeout.verify` and all 413 recorded measurements re-simulate on your own machine.

Status, stated plainly: the live Nebius Token Factory path is tested end to end against a protocol-faithful mock. Real-model runs are labelled LIVE and offline stand-ins are labelled FALLBACK, both on screen and in the datasheet's provenance table.

Inspiration

LLM agents can now draft analog circuits, but a generated datasheet is a claim, not evidence. Every demo shows a green checkmark at one operating point: typical silicon, nominal supply, room temperature. Real chips get judged at their corners. We wanted the agent that does the engineer's hardest job: trying to break the claim before anyone else does.

The real problem isn't generating datasheets; AI can already do that. It's that **nobody stands behind the numbers**. Tapeout is a generator whose deliverable records: - who broke which promise - under what conditions - with which simulation as proof

That matters for: - **Hardware reliability:** the gap between promise and behaviour becomes auditable. - **AI reliability:** every model claim is judged by an outside simulator, not by the model itself. - **EDA workflows:** every counterexample becomes a permanent regression test.

What it does

Type a product ("a heart-rate monitor front-end for a fitness band") and click once. The run then goes through six stages:

1. **Scout** (Tavily + Nemotron Nano) researches the product and keeps only facts whose source URL it actually retrieved. It also finds a real commercial op-amp for the same job and puts its datasheet numbers next to ours. A figure is kept only if it appears verbatim in the page Tavily retrieved; anything else the model reports is discarded and disclosed.
2. **Architect** (Nemotron Ultra) plans the chip: amplifier specs with units, plus logic cells from a fixed menu. A deterministic compiler rejects infeasible plans and the model repairs them. No model ever writes a netlist.
3. **Fab:** Nemotron Super races differential evolution to size each amplifier against ngspice and SkyWater 130nm foundry models. Logic blocks are proved against their full truth table by transient simulation.
4. **Red team** (Nemotron Ultra) reads the sizing and margins and predicts the conditions where each claim should fail: process corner, supply, temperature, common mode, load. A deterministic lab tests each prediction and bisects it to the exact boundary. A 45-point corner grid audits the predictions.
5. A claim that breaks inside the operating envelope goes back to the design team for a multi-corner repair, then is attacked again. Every earlier counterexample stays as a regression test. A break outside the envelope is published as a limit.
6. **Writer** (Nemotron Nano) drafts the datasheet. An auditor strikes any number that no simulation backs.

The result is a datasheet with a **robustness envelope**, a **falsification log**, and a **shmoo plot**: pass/fail over supply × temperature for every process corner. Shown before and after repair, it shows red cells inside the operating box turning green. Click any number to see the exact SPICE deck and raw ngspice output behind it.

How we built it

The model proposes; the simulator judges. The tiers are split by what each task needs: - **Nemotron Ultra** makes the two decisions where reasoning depth changes the outcome: the chip plan and the failure hypotheses. - **Nemotron Super** runs the latency-bound inner loop with the simulator, 10–20 sizing calls per block. - **Nemotron Nano** does fast extraction and prose.

No model output is accepted on its word. Every hypothesis is tested against ngspice and bisected 5 times to its exact boundary, and every surviving claim is re-attacked after each repair.

- **Models:** Nebius Token Factory via its OpenAI-compatible streaming API, with three Nemotron tiers split by job:
- Ultra for the two reasoning-heavy decisions, the plan and the failure hypotheses.
- Super for the latency-bound inner loop with the simulator.
- Nano for extraction and prose.
- **Engine:** our own circuit engine (deterministic synthesis, netlist validation, topologies) on ngspice with SkyWater sky130 corner models.
- **Forks:** each stress run is a fork of a frozen, content-hashed design, run in its own ngspice process. Failures become leaves in the log and never mutate the design.
- **Front end:** FastAPI server-sent events stream every token, simulation and verdict live to a single-page UI.
- **Honesty:** every agent declares whether it ran live on Nemotron or fell back, on screen and on the datasheet.

Challenges we ran into

Every serious problem was a bug the falsification loop exposed in our own work:

1. **The testbench lied.** The original engine measured amplifiers open-loop with no DC feedback, so a 5% supply droop made gain "collapse" from 63 dB to 28 dB. The falsifier would have published fake failures. A DC servo (a 1 GH inductor into a 1 F cap) pins the operating point and keeps the AC measurement open-loop. It also showed the real sky130 two-stage gain ceiling is about 82 dB, not the 65 dB the railed bench suggested.
2. **Repairs targeted the wrong corners.** The first repair loop took its conditions from the first few grid failures. Meanwhile GBW had collapsed to 76 kHz at sf / 1.71 V / −40 °C and phase margin to 23° at fs / 1.89 V / 85 °C. Repairs now target each broken claim's worst in-envelope run.
3. **A repair reopened an old failure.** Repair 2 fixed round 2's corners and broke round 1's again; the re-attack caught it. Every counterexample is now a permanent regression test that every later repair must pass.

Integration surprises came out of a protocol-faithful Token Factory mock: - `<think>` tags split across stream chunks swallowed the architect's answer. - A sizing loop could spend thousands of calls re-proposing sizings it had already tried.

Both are fixed and covered by tests.

Best Use of Tavily

Why Tavily is load-bearing: this is citation-enforced generation, not RAG. - A commercial reference figure survives only if it appears verbatim, with the right unit, in the page Tavily retrieved. - When the model invented "140 dB", it was discarded, and the datasheet says so. - Tavily's retrieval is the audit trail that makes the reference column trustworthy.

Tavily does two jobs, and neither is decoration: - **Grounding the design.** The scout's search results become cited facts the architect plans from. A fact whose URL was not actually retrieved is dropped. - **Benchmarking it.** A second search finds the commercial op-amp a product team would otherwise buy. Its datasheet figures appear in a "Commercial ref." column next to our simulated ones, each verified verbatim against the retrieved page.

That turns "we designed an op-amp" into "here is the part you would buy, and here is ours".

Accomplishments that we're proud of

- A datasheet whose limits are found by simulation, with every number traceable to the run that produced it.
- No silent fallbacks: if Nemotron didn't do something, the page says so.
- 346 tests. CI builds the container and proves a sky130 simulation inside it.
- **The evidence re-runs anywhere.** `python -m tapeout.verify` re-simulates all 83 stored decks of the reference run on the checker's own machine. 413 of 413 recorded measurements reproduce; a doctored number is caught. CI does this on every push.
- A recorded clean-room run (fresh clone → 340 tests passing in under 2 minutes) is in `docs/CLEANROOM.md`.
- A judge can verify the simulation path with no keys at all: `python -m tapeout.check --offline`. The static replay page needs nothing.

What we learned

Verification has to be independent of the agent that designed the thing. The most useful thing an LLM did here was propose *where to look for failure*. The simulator decides whether the failure is real.

What's next

- Noise, slew-rate, PSRR and Monte Carlo mismatch analyses.
- More topologies.
- Running forks as Nebius Serverless Jobs for large corner campaigns.
- Exporting the robustness envelope in a standard datasheet format.

Pre-existing work

The circuit engine (`bias/`) comes from our earlier project, cadence_auto. Everything in `tapeout/` and `web/` was built during the submission period:

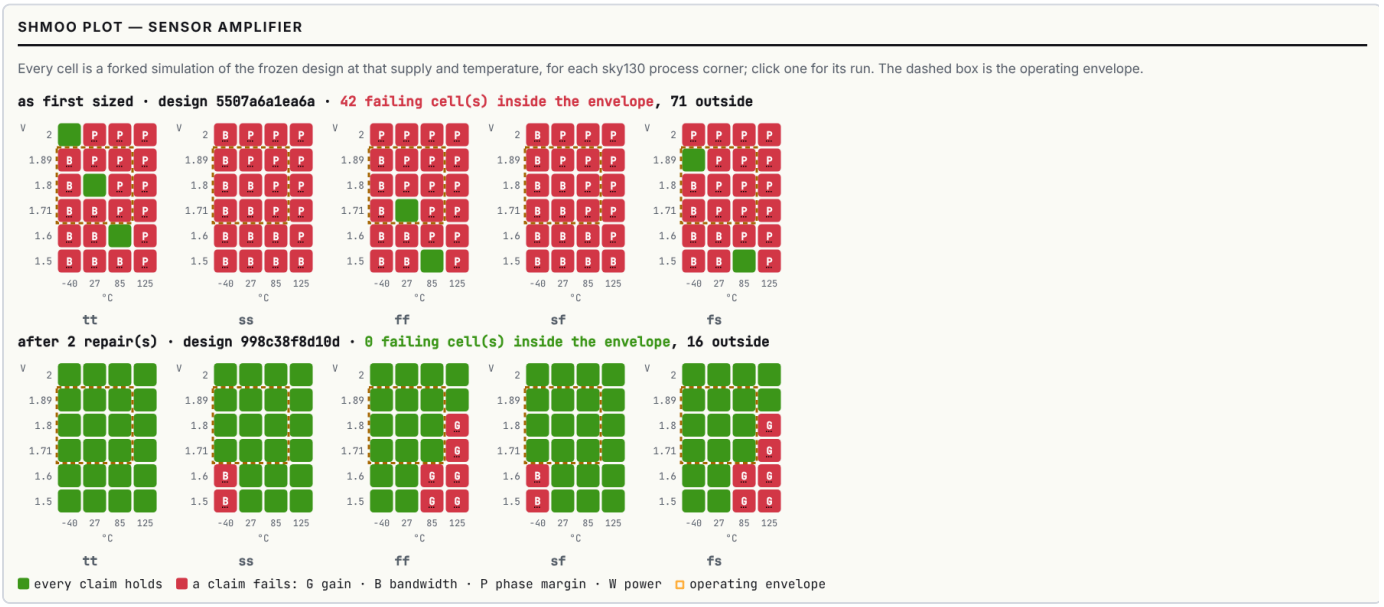
- the Nemotron/Token Factory agents
- Tavily research
- the compiler repair loop and the sizing race
- the falsification engine, multi-corner repair and regression tests
- the auditor, the provenance labelling and the UI

In `bias/`, the DC-servo testbench and process-corner forking are also new.

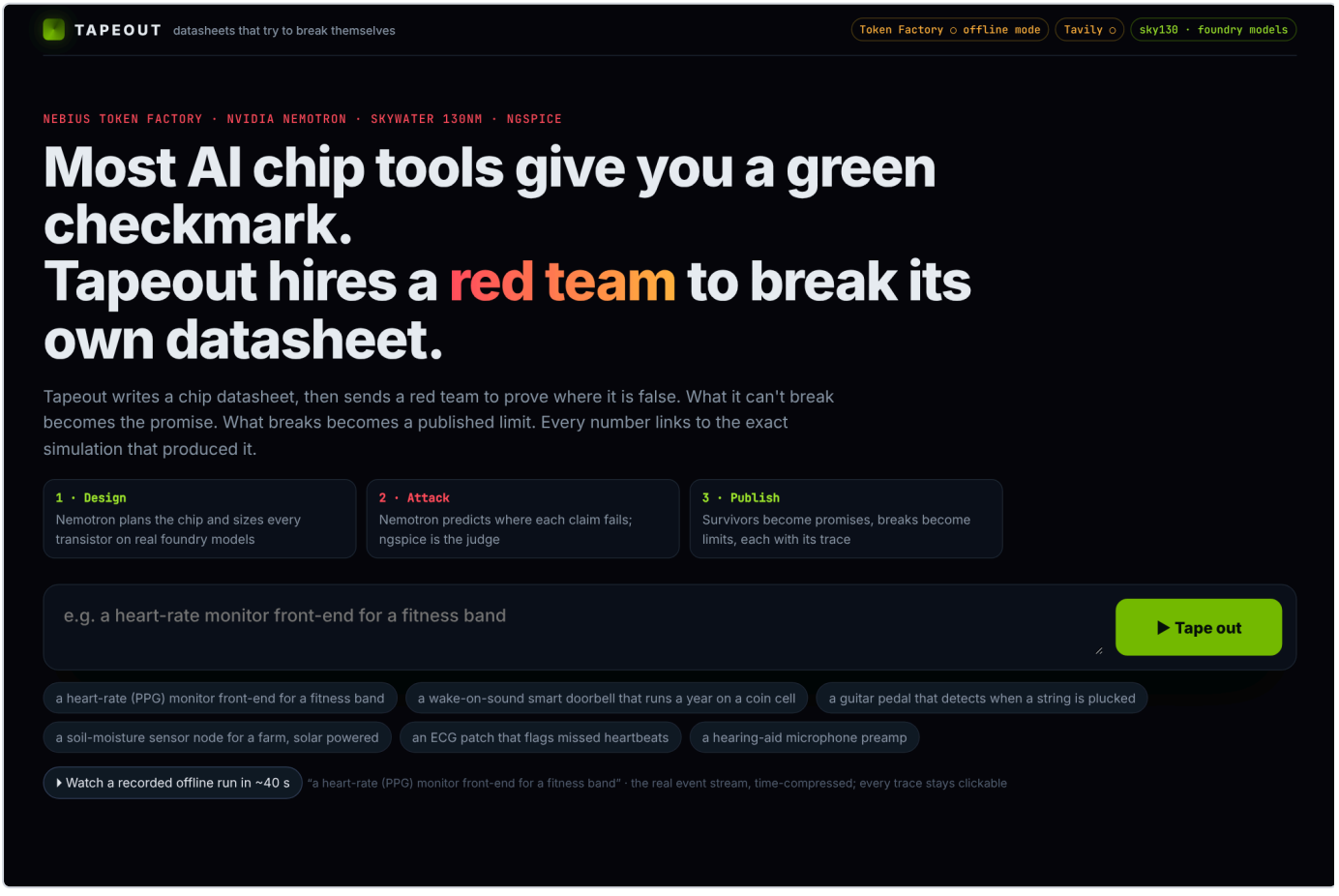
Links

- **Code (public, MIT):** https://github.com/Amritha902/NEBIUS
- **Demo (instant replay):** see "Try it out" links
- **Verify the evidence yourself:** `python -m tapeout.verify web/replays/ppg-offline.jsonl`

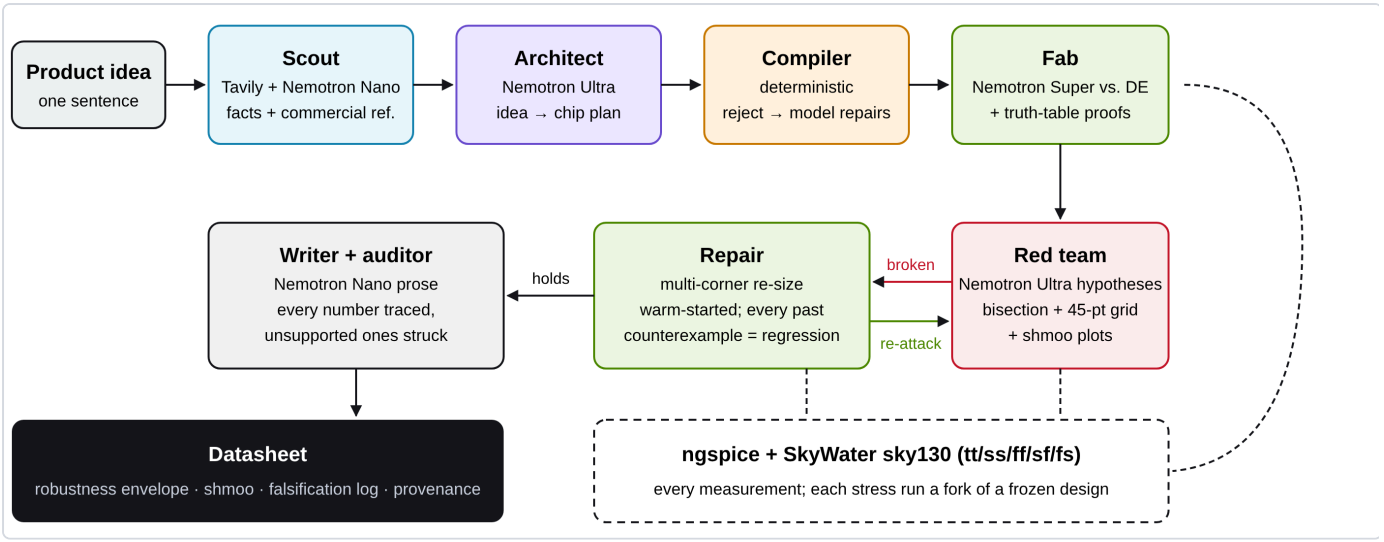
3. Image gallery: upload in this order



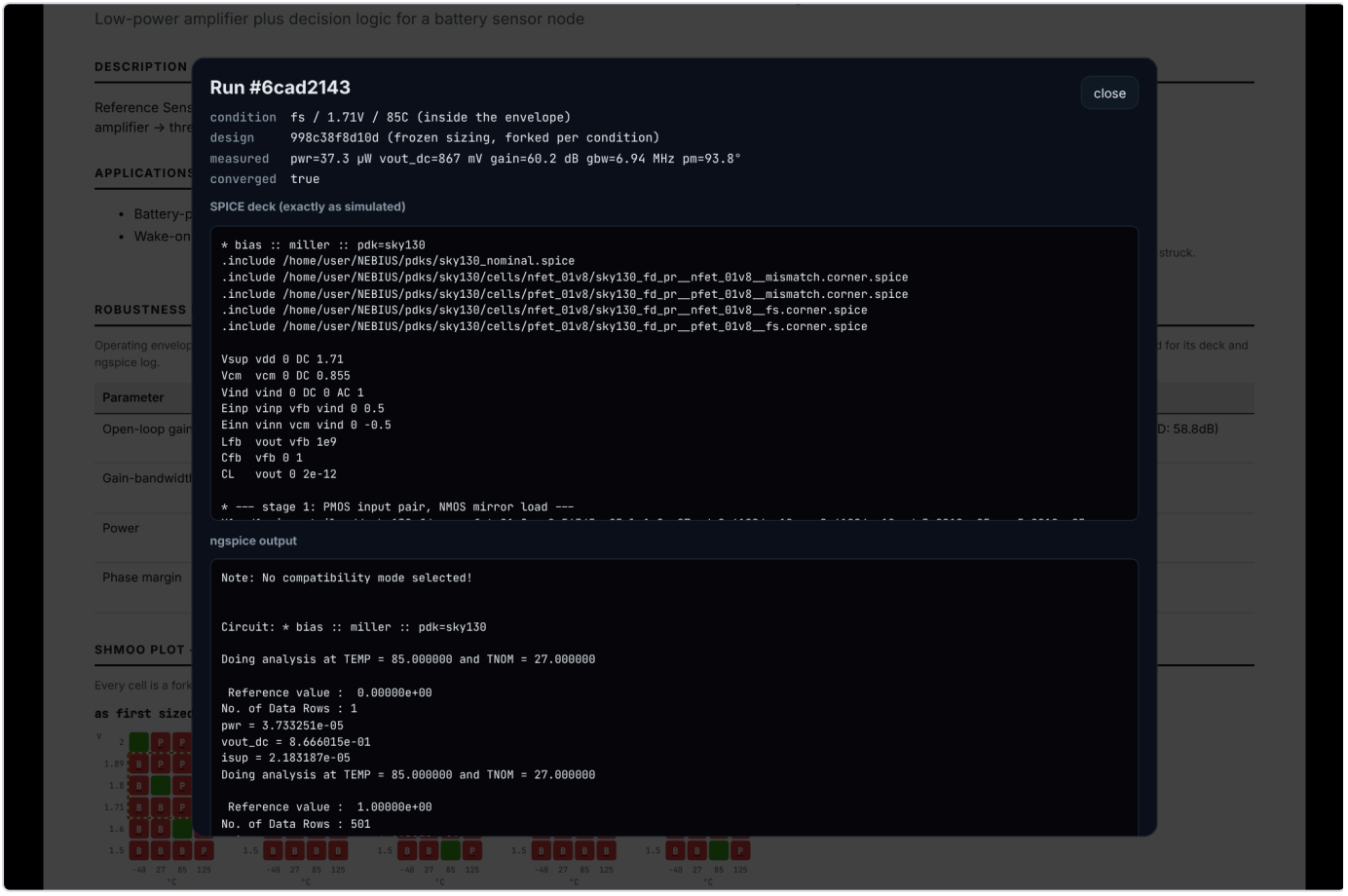
1. images/shmoo.png: Cover image. Shmoo plot: pass/fail over supply × temperature for every process corner, before vs. after repair (42 → 0 failures inside the envelope).



2. images/hero.png: The app: one sentence in, then Design → Attack → Publish.



3. images/architecture.png: Pipeline: Scout → Architect → Compiler → Fab → Red team ⇄ Repair → Writer → Datasheet, with ngspice as the judge.



4. images/trace.png: Click any number: the exact SPICE deck and the raw ngspice log behind it.

ROBUSTNESS ENVELOPE — SENSOR AMPLIFIER

Operating envelope: all five sky130 process corners (tt, ss, ff, sf, fs), VDD 1.71–1.89 V, -40 to 85 °C, input common mode 0.4–0.6·VDD, 2 pF load. 327 stress simulations over 3 round(s). Click any run id for its deck and ngspice log.

Parameter	Claim	Typical	Worst case in envelope	Status	Limits found outside the envelope
Open-loop gain	⩾ 60.0 dB	63.8 dB	60.2 dB fs / 1.71V / 85C #6cad2143	HOLDS	gain holds to vcm=0.80*VDD and breaks at 0.81*VDD (tt / 1.8V / 27C / Vcm=0.81*VDD: 58.8dB)
Gain-bandwidth	⩾ 2 MHz	9.22 MHz	3.23 MHz sf / 1.71V / -40C #b35cb366	HOLDS	gbw holds to vdd=1.603V and breaks at 1.594V (ss / 1.59V / -40C: 1.912MHz)
Power	⩽ 200 µW	38.9 µW	41.4 µW fs / 1.89V / 85C #52888d49	HOLDS	none found
Phase margin	⩾ 55.0°	97.0°	92.3° ss / 1.71V / 85C #17bb0f18	HOLDS	none found

5. **images/robustness.png**: Robustness envelope: claim, typical, worst case in envelope, status, limits found outside.

PROVENANCE

Agent	Ran on	Detail
scout	FALLBACK	TAVILY_API_KEY not set; architect will rely on its own knowledge
architect	FALLBACK	no NEBIUS_API_KEY
fab:afe	FALLBACK	offline: no NEBIUS_API_KEY
redteam:afe	FALLBACK	0 of 6 hypotheses from Nemotron; 6 textbook stand-ins (no NEBIUS_API_KEY)
writer	FALLBACK	template prose (no NEBIUS_API_KEY)

6. **images/provenance.png**: Provenance: which agent ran live on Nemotron and which was a fallback (from the offline run; re-capture after a live run).

FALSIFICATION LOG — SENSOR AMPLIFIER

Round 1 · design 5507a6a1ea6a · nominal #bfde7114 · 13 targeted sims + 39 new grid sims (45 corners; repeats reuse earlier runs)

#	Hypothesis	Verdict	Evidence
H1	**pm** — A heavier load pulls the output pole down toward the GBW; the fast corner raises GBW at the same time. by textbook heuristic · push cl_x → 6 from {"corner":"ff"}	BROKEN pm fails at ff / 1.8V / 27C: measured 47.4deg	#ea8b93ec
H2	**pm** — Hot silicon shifts the second-stage pole relative to the compensation zero. by textbook heuristic · push temp → 150 from {"corner":"fs"}	BROKEN pm fails at fs / 1.8V / 27C: measured 41.8deg	#adc0d4ec
H3	**gbw** — Slow devices and a cold, low supply cut gm of the input pair. by textbook heuristic · push vdd → 1.5 from {"corner":"ss","temp":-40}	BROKEN gbw fails at ss / 1.8V / -40C: measured 202.1kHz	#ab8ebaa1
H4	**gain** — The input pair leaves saturation as the common mode nears a rail. by textbook heuristic · push vcm → 0.9 from {}	LIMIT gain holds to vcm=0.72*VDD and breaks at 0.74*VDD (tt / 1.8V / 27C / Vcm=0.74*VDD: 56.2dB)	#bfde7114 #f268f739 #7816cfdd #d92ea515 #a893cacf #57e2b2b5 #bb6a3d1e
H5	**pwr** — Fast devices on a high supply draw more bias current. by textbook heuristic · push vdd → 2 from {"corner":"ff","temp":-40}	SURVIVED pwr holds from ff / 1.8V / -40C all the way to vdd=2.000V	#e334b332 #c2476b31
H6	**gain** — Output resistance falls at high temperature. by textbook heuristic · push temp → 150 from {"corner":"ss"}	SURVIVED gain holds from ss / 1.8V / 27C all the way to temp=150C	#29cef4c4 #d718298b

Grid: gbw violated at 19 of 45 corners (#f67459dc #33810f72 #478a52d5); pm violated at 23 of 45 corners (#818a16df #e4d0c5a7 #757a0928).

Repair 1: re-sized by evolution — re-sized against nominal + sf / 1.71V / -40C; fs / 1.89V / 85C.

Round 2 · design fa916125a799 · nominal #4544fb79 · 26 targeted sims + 39 new grid sims (45 corners; repeats reuse earlier runs)

#	Hypothesis	Verdict	Evidence
H1	**pm** — A heavier load pulls the output pole down toward the GBW; the fast corner raises GBW at the same time. by textbook heuristic · push cl_x → 6 from {"corner":"ff"}	LIMIT pm holds to cl_x=1.47x load and breaks at 1.62x load (ff / 1.8V / 27C / CL=3.25pF: 54.6deg)	#17ad309d #2c3f9368 #9421ff3e #07ecb20d #01777866 #70f7fb21 #c6ce69d1
H2	**pm** — Hot silicon shifts the second-stage pole relative to the compensation zero. by textbook heuristic · push temp → 150 from {"corner":"fs"}	SURVIVED pm holds from fs / 1.8V / 27C all the way to temp=150C	#aad686f6 #1f5da6e7
H3	**gbw** — Slow devices and a cold, low supply cut gm of the input pair. by textbook heuristic · push vdd → 1.5 from {"corner":"ss","temp":-40}	LIMIT gbw holds to vdd=1.584V and breaks at 1.575V (ss / 1.57V / -40C: 1.89MHz)	#fb279344 #985c4a27 #0b1f00c9 #81ee3e56 #d04fa2bb #edc41a39 #86153550
H4	**gain** — The input pair leaves saturation as the common mode nears a rail. by textbook heuristic · push vcm → 0.9 from {}	LIMIT gain holds to vcm=0.81*VDD and breaks at 0.83*VDD (tt / 1.8V / 27C / Vcm=0.83*VDD: 59.8dB)	#4544fb79 #dfee86e4 #593cdcc6 #d6935de4 #658501b6 #c9e956cf #59fb652d
H5	**pwr** — Fast devices on a high supply draw more bias current. by textbook heuristic · push vdd → 2 from {"corner":"ff","temp":-40}	SURVIVED pwr holds from ff / 1.8V / -40C all the way to vdd=2.000V	#c6a42c51 #8f875f75
H6	**gain** — Output resistance falls at high temperature. by textbook heuristic · push temp → 150 from {"corner":"ss"}	SURVIVED gain holds from ss / 1.8V / 27C all the way to temp=150C	#16e3181c #93184b3f

Grid: gain violated at 2 of 45 corners (#cf44f2f0 #98912f2d); pm violated at 3 of 45 corners (#1bccee18 #88b22afe #741b7759).

Repair 2: re-sized by evolution — re-sized against nominal + sf / 1.71V / -40C; fs / 1.89V / 85C; ff / 1.71V / 85C; ss / 1.8V / 85C.

Round 3 · design 998c38f8d10d · nominal #a5d19a7b · 21 targeted sims + 39 new grid sims (45 corners; repeats reuse earlier runs)

#	Hypothesis	Verdict	Evidence
H1	**pm** — A heavier load pulls the output pole down toward the GBW; the fast corner raises GBW at the same time. by textbook heuristic · push cl_x → 6 from {"corner":"ff"}	SURVIVED pm holds from ff / 1.8V / 27C all the way to cl_x=6.00x load	#b8af5d16 #cfa63327
H2	**pm** — Hot silicon shifts the second-stage pole relative to the compensation zero. by textbook heuristic · push temp → 150 from {"corner":"fs"}	SURVIVED pm holds from fs / 1.8V / 27C all the way to temp=150C	#4a0f01f0 #3c498af6
H3	**gbw** — Slow devices and a cold, low supply cut gm of the input pair. by textbook heuristic · push vdd → 1.5 from {"corner":"ss","temp":-40}	LIMIT gbw holds to vdd=1.603V and breaks at 1.594V (ss / 1.59V / -40C: 1.912MHz)	#68847bca #a5aecf5d #ced308cf #eba7c23f #829c5ae1 #aae1396c #c2eee0ec
H4	**gain** — The input pair leaves saturation as the common mode nears a rail. by textbook heuristic · push vcm → 0.9 from {}	LIMIT gain holds to vcm=0.80*VDD and breaks at 0.81*VDD (tt / 1.8V / 27C / Vcm=0.81*VDD: 58.8dB)	#a5d19a7b #766e610e #8f0921f7 #3b630f86 #3b7b54ba #560b8728 #1d667480
H5	**pwr** — Fast devices on a high supply draw more bias current. by textbook heuristic · push vdd → 2 from {"corner":"ff","temp":-40}	SURVIVED pwr holds from ff / 1.8V / -40C all the way to vdd=2.000V	#1871db73 #8f4b9af3
H6	**gain** — Output resistance falls at high temperature. by textbook heuristic · push temp → 150 from {"corner":"ss"}	SURVIVED gain holds from ss / 1.8V / 27C all the way to temp=150C	#4be5b30e #4c8fa42c

Grid: no violation at any of 45 corners.

7. images/falsification-log.png: Falsification log: rounds, hypotheses, verdicts, repairs.

4. Demo video

File	`video/Tapeout-demo.mp4` : 2 min 12 s, 1920×1080 H.264 with AAC audio, 32 MB
Content	Animated opening ("every AI chip demo shows you a pass") → the problem → an animated three-tier Nemotron architecture with ngspice as the judge → the app → the live replay (agents, sizing race, red-team cards) → animated round-1 failure counters → shmoo plot before/after → clicking a number to its SPICE deck and ngspice log → robustness and falsification log → provenance table → a `tapeout.verify` terminal → end card with the repo link
Voice	An AI voiceover (Kokoro, open-source neural text-to-speech, run locally), with matching subtitles and a quiet synthesized music bed. It says "NVIDIA Nemotron" and "Nebius Token Factory", and states at 0:39 that the screens are a recorded offline run.
Upload	youtube.com → Create → Upload video → choose the file → title "Tapeout: a chip datasheet that tries to prove itself wrong (Nebius × NVIDIA hackathon)" → Visibility: **Public** → Publish → copy the link into Devpost

Narration with timestamps

```
VOICEOVER for video/Tapeout-demo.mp4 (2 min 12 s, 1920x1080, AI voiceover + subtitles)

The narration is AI-generated with Kokoro, an open-source neural text-to-speech
model (voice "af_heart"), run locally. Subtitles show the same words. The music
bed is synthesized. The screens are a recorded OFFLINE run of the app:
built-in stand-ins replace Nemotron there, and every simulation is real. The
video says this out loud at 0:39.

0:01.3  Every AI chip demo shows you a circuit that passes. Tapeout shows you where it breaks,
and proves it.
0:08.0  AI can already write a datasheet. The problem is, nobody stands behind the numbers. So
Tapeout writes the datasheet, then sends a red team to prove where it's false.
0:19.3  Three NVIDIA Nemotron tiers, served by Nebius Token Factory. Ultra plans the chip and
predicts how it will fail. Super sizes every transistor. Nano researches, and writes. But the
judge is not a model. The judge is N G spice, running real SkyWater foundry models.
0:38.8  One sentence in. One click. This is a recorded offline run, so built-in stand-ins
replace the models here. But every simulation you see is real.
0:49.4  The agents stream live. The fab races Nemotron against an evolutionary optimizer to size
the amplifier. Then the red team attacks. Each card is a predicted failure mechanism. Green
means the claim held. Red means it broke.
1:05.1  Round one. Phase margin collapses to twenty three degrees, against a fifty five degree
claim. Bandwidth falls to seventy six kilohertz. So the design goes back for repair, and gets
attacked again.
1:18.1  The shmoo plot tells the whole story. Forty two failures inside the operating envelope
as first designed. Zero, after two repairs. The red that remains sits outside the envelope, and
is published as a limit.
1:32.5  And every number links to the exact spice deck, and the raw simulator log, that produced
it.
1:38.9  You get a robustness envelope, and a falsification log, where every counterexample
becomes a permanent regression test.
1:46.7  A provenance table declares which agent ran live on Nemotron, and which was a fallback.
Nothing is passed off as live.
1:55.5  Don't take our word for it. One command re-simulates all four hundred and thirteen
recorded measurements, on your own machine.
2:03.6  Tapeout. Limits proven, not claimed. Built with NVIDIA Nemotron, on Nebius Token
Factory.
```

5. Feedback survey answers

```
FEEDBACK SURVEY ANSWERS (Most Valuable Feedback prize)
======================================================

Nebius Token Factory
- The OpenAI-compatible streaming API made the live agent UI cheap to build:
  the browser console is just the token stream re-emitted. Usage arrives in
  the final chunk (stream_options.include_usage), so per-model cost
  accounting was trivial.
- Model ids are case-sensitive, and Nemotron 3 ids were hard to confirm from
  public docs. We resolve them at startup from GET /v1/models. Request: a
  stable alias per tier (e.g. nemotron-ultra-latest) and one docs page
  listing exact ids.
- Request: a documented "thinking off" switch per model. We used
  chat_template_kwargs {"enable_thinking": false} for fast JSON replies, but
  found it by trial and error.

NVIDIA Nemotron
- Tier split works as advertised:
  - Ultra for the two decisions where reasoning depth changes the outcome
    (the chip plan, and predicting where a circuit fails).
  - Super for a fast inner loop with a simulator (10-20 short JSON calls per
    block).
  - Nano for extraction and prose.
- Thinking output varies by model, either a reasoning_content delta or inline
  <think> tags. Tags split across stream chunks broke our first parser. A
  single documented field across the family would remove that code.
- Request: a JSON-mode / schema guarantee per model, so agent loops don't
  need brace-matching fallbacks.

Tavily
- Search results with page content let us do citation-enforced generation:
  a figure the model reports is kept only if it appears verbatim in a page
  Tavily retrieved. That turned Tavily into an audit trail, not just a
  search box.
```

6. Final checklist

- [ ] GitHub Pages on (`main`, `/docs`), and the default branch set to `main`
- [ ] Claude demo link shared publicly (or left out)
- [ ] Video uploaded to YouTube as **Public**; link pasted
- [ ] Name, pitch, description, built-with, track filled in
- [ ] Best Use of Tavily ticked; feedback survey filled in
- [ ] 7 images uploaded, `shmoo.png` first
- [ ] "Existed before?" answered with the cadence_auto paragraph
- [ ] **Submit** before 10:30 PM IST, then open the public project page to check the links